CTI / v5.2.0.3

Clinical Trial Intelligence Workbench

Full operating & testing guide

Windows with Docker Desktop — PostgreSQL included

Source data, saved predictions and guided explanations

All 26 sections · 37 acceptance checks · explicit evidence limits

SET UP · IMPORT · PREDICT · REVIEW

Research decision support with source and model provenance

Editable Markdown accompanies this PDF. Use the PDF chapter bookmarks.

Historical approved models, raw records and metrics remain preserved.

Not clinical or regulatory advice. Final release evidence has its own report.

Docker operating manual edition · 14 October 2026

Contents

Begin with Windows Docker startup in chapter 2. Chapters 6–14 explain data and research use; chapters 15–22 cover testing, every section, files and evidence. Commands are provided in the accompanying Markdown for exact copying.

1. What to open first . . . 3
2. Windows startup with Docker Desktop . . . 4
3. Native alternatives and Linux/macOS . . . 6
4. Environment values . . . 8
5. Projects, roles and invitations . . . 10
6. Import and review data . . . 11
7. Overview graphs and study comparison . . . 12
8. Saved-model prediction . . . 13
9. Seven visual explanations . . . 14
10. Scenarios and planning . . . 15
11. Train, compare, approve and activate . . . 16
12. Protocols, sources and formulas . . . 17
13. Assistant configuration and feedback . . . 19
14. Settings and security . . . 20
15. Automated and manual tests . . . 21
16. CLI research and release builds . . . 22
17. Backup and deployment . . . 23
18. Troubleshooting and evidence limits . . . 24
19. All sections and the 37-check acceptance workflow . . . 25
20. Module and file map . . . 28
21. What changed from v5.2.0.2 . . . 30
22. Verification status and downloadable materials . . . 31
23. Actual application screenshots . . . 33

1. What to open first

The ZIP includes the built React interface, FastAPI source, PostgreSQL configuration, tests, the frozen 800-record public registry cohort, verified saved models and this operating guide. PostgreSQL holds accounts, sessions, projects, memberships, chat, feedback and research metadata. Source files and model artifacts remain on persistent disk. There is no SQLite fallback.

Start in this order: prepare the environment; start PostgreSQL and the workbench; create an owner account; create/open a project; load the starter cohort; inspect Overview; use Prediction Studio. Training is an independent owner workflow and is not required for the bundled predictions.

The main navigation is Overview, Studies, Prediction Studio, Protocol Evidence, Scenarios, Models, Assistant and Settings. More tools contains Data Manager, Experiments, Validation Lab, the evidence animation pages, Phase Academy, Research sources, Saved library and Access Control.

2. Windows startup with Docker Desktop

This edition prioritizes Docker because Docker is installed on your computer. Docker runs both PostgreSQL and the application, with persistent named volumes. The app remains v5.2.0.3; datasets, approved models and research workflows are unchanged.

Start Docker and check Python

1. Open Docker Desktop. Wait until the engine is running. Use Linux containers; complete any WSL2 configuration or restart Docker requests.
2. Confirm the Windows Python launcher can find Python 3.12: `py -3.12 --version`. If needed, install Python 3.12 with its launcher from python.org.
3. You do not need a separate PostgreSQL installation or Node on Windows for this path. Host Python initializes the private settings and launches Docker Compose. Application dependencies are installed inside the image.

Extract and start the full application

1. Right-click the complete ZIP → Extract All into a writable folder such as `C:\CTI`. Open the folder containing `START_DOCKER_WINDOWS.bat`; do not run launchers inside the ZIP viewer.
2. Double-click `START_DOCKER_WINDOWS.bat`. If you already ran `SETUP_WINDOWS.bat`, this is your next step; there is no need to repeat setup or configure a native PostgreSQL service.
3. Let the first download/build finish. The launcher prepares `.env` without displaying credentials, preserves existing settings, builds the UI/backend and starts the PostgreSQL and workbench services.
4. Wait for **Application ready: http://localhost:8000**. The launcher opens the browser; open that address yourself if required. Keep Docker Desktop running while using the application.
5. Open your private `.env` in a text editor. Copy the value after `SETUP_TOKEN=` into the first-owner form. Enter your name, email and a unique password of at least 12 characters. If the first owner was already created, sign in instead. Keep `.env` private.
6. Create/open a project. More tools → Data Manager → load the 800-record starter cohort and select it. Review Overview and use Prediction Studio. Bundled approved predictions do not require training.

Equivalent commands from PowerShell in the extracted project folder:

```
.\START_DOCKER_WINDOWS.bat
```

Or run the underlying steps explicitly:

```
py -3.12 scripts/initialize_env.py
docker compose up -d --build workbench
docker compose ps
```

The app opens at `http://localhost:8000`. A healthy PostgreSQL service is required before the app starts. The first build may take several minutes depending on Internet speed and hardware. Do

not run the native application on the same port at the same time.

Later starts, stops and provider settings

Open Docker Desktop, then run the same launcher. Compose reuses valid cached layers; it does not reset your volumes or owner account. `docker compose down` stops the services and keeps named volumes. Do not remove named volumes unless you intend to delete their data. Back up PostgreSQL and research file storage together, as described in chapter 17.

After editing provider keys/model IDs in `.env`, run:

```
docker compose up -d --force-recreate workbench
```

This recreates the container with the updated environment while preserving named volumes. `docker compose restart` alone does not load changed environment variables.

If startup fails

Run `docker version`, `docker compose version`, `docker compose ps` and `docker compose logs --tail 100 workbench postgres`. If the engine is unavailable, open Docker Desktop and wait for it. If port 5432 is already in use, set `POSTGRES_PORT=5434` in `.env` and rerun. Container-to-container PostgreSQL still uses `postgres:5432`. If port 8000 is occupied, stop the native workbench or other process using it before starting Docker. Preserve existing database passwords and connection URLs; changing an environment password does not reset an existing PostgreSQL volume's role password.

3. Native alternatives and Linux/macOS

Optional native Windows startup

Use the full ZIP on a Windows computer. Python 3.12 and native PostgreSQL are required. Docker and Node are unnecessary when using the included prebuilt interface. Internet is needed to install Python dependencies and for optional live sources/assistant providers.

Install the prerequisites

1. Download Python 3.12 from the official Python Windows downloads page. Select a Python 3.12 Windows installer matching your computer. Include the Python launcher; the launchers use `py -3.12`. Adding Python to PATH is convenient but does not replace the launcher.
2. Download PostgreSQL through the official PostgreSQL Windows page. Install the server and command-line tools. pgAdmin is optional for this workbench.
3. During PostgreSQL installation choose a strong password for its administrator role `postgres`. Remember it privately. Keep the selected port, usually 5432. This password is separate from the application database password and your workbench account password.
4. In Windows Services confirm the installed PostgreSQL service is running. The application will connect to it; the workbench launcher does not install or start that Windows service.

Extract, set up, connect and start

1. Download the complete v5.2.0.3 Workbench ZIP. Right-click → Extract All into a writable local directory such as `C:\CTI`. Open the extracted directory containing `SETUP_WINDOWS.bat`; do not run inside the ZIP viewer.
2. Double-click `SETUP_WINDOWS.bat` once. Wait for successful dependency installation. It creates `.venv`, prepares your private `.env` and generates application credentials plus the first-owner `SETUP_TOKEN`.
3. Double-click `CONNECT_DATABASE_WINDOWS.bat`. For a new local installation press Enter for host `127.0.0.1`, port `5432`, application role `cti` and application database `cti`. Press Enter at the application-password prompt to keep the generated private password. If your PostgreSQL installation uses another port, enter it.
4. Accept administrator role `postgres` and maintenance database `postgres`. Enter the administrator password chosen during PostgreSQL installation at the hidden prompt. The helper uses Python `getpass`; password characters are not displayed and the administrator password is not saved.
5. Wait for **Application database login verified.** The helper creates only missing roles/databases, leaves existing role passwords and database contents intact, and writes `DATABASE_URL` only after verifying application login. It does not reset data or run application schema operations.
6. Double-click `START_WINDOWS.bat`. Leave the terminal open and visit `http://localhost:8000`, or the local address/port printed by your configured server. The application performs its normal initialization on start.
7. Open your local `.env` in a text editor and copy only the value after `SETUP_TOKEN=` into the first-owner form. Enter your name, email and a unique account password of at least 12 characters. Do not share `.env` or paste its full contents into chat.

8. Create a project, open it, choose More tools → Data Manager and load the 800-record starter cohort. Select that dataset. Review Overview and run Prediction Studio with a bundled approved model.

Equivalent PowerShell commands, from the extracted project root:

```
py -3.12 scripts/setup.py
.venv\Scripts\python.exe scripts/configure_postgres.py
.venv\Scripts\python.exe scripts/start.py
```

Subsequent starts and existing installations

On later visits keep PostgreSQL running and use `START_WINDOWS.bat`; repeat setup only to install/refresh dependencies. Close the server with Ctrl+C when finished. Stored projects remain in PostgreSQL and source/model files remain under persistent storage.

`CONNECT_DATABASE_WINDOWS.bat --check` performs a read-only login check with no credential prompts or file changes. `--configure` deliberately selects other connection settings. A working existing URL is verified and preserved without administrator prompts. Unavailable remote URLs are preserved; explicit remote reconfiguration requires typing `CONFIGURE`. Existing URL options, including TLS/certificate settings, remain defaults.

If the application role already exists with another password, the helper refuses to reset it. Enter its known password or choose a new role. Creation needs an administrator with CREATEROLE and CREATEDB. Partial creation can leave a missing object created before a later failure; retry safely reuses it. Only `DATABASE_URL` changes after success; unrelated secrets, `COMPOSE_DATABASE_URL` and `SETUP_TOKEN` are retained.

Native Python and PostgreSQL on Linux/macOS

Install Python 3.12 and a running PostgreSQL service. From the extracted project root:

```
python3.12 scripts/setup.py
.venv/bin/python scripts/configure_postgres.py
.venv/bin/python scripts/doctor.py
.venv/bin/python scripts/start.py
```

The connector verifies login and creates only missing application role/database objects. It preserves existing passwords, data, connection URLs and TLS options. Node is unnecessary with the prebuilt interface.

Docker on Linux/macOS

Install Docker with Compose, then run `python3.12 scripts/initialize_env.py --start` from the extracted project root. Docker starts PostgreSQL and the workbench using the same named volumes. Keep TLS verification enabled when building and downloading dependencies.

Frontend development only

Install Node.js 22, run `npm ci` then `npm run dev` inside `frontend/`, with the backend running on port 8000. `npm run build` regenerates `frontend/dist`. Packaged app users do not need host Node. `scripts/setup.py --dev` installs native Python test dependencies; use a separate disposable test database as described in chapter 15.

4. Environment values

Initialization prepares a private `.env` from `.env.example`, generating local database credentials and a setup token while preserving existing operator values. Credentials are excluded from release ZIPs. The native connector writes only a verified `DATABASE_URL`.

Use secrets only in your local server environment or managed secret store. Do not use `VITE_` variables for credentials: frontend variables can become public browser assets. Restart the server after environment changes. Never publish `.env`, passwords, provider keys, sessions, runtime logs or databases.

Variable	Purpose
`POSTGRES_USER`, `POSTGRES_DB`, `POSTGRES_PASSWORD`	Local Docker database credentials; the password is generated.
`POSTGRES_PORT`	Published local Docker/native database port, normally 5432.
`DATABASE_URL`	Native application's PostgreSQL connection.
`COMPOSE_DATABASE_URL`	Container PostgreSQL connection, normally using hostname `postgres`.
`TEST_DATABASE_URL`	Disposable test database, normally port 5433 and database `cti_test`.
`SETUP_TOKEN`	Generated first-owner setup secret.
`APP_ENV`, `AUTH_MODE`	Development/production environment; use authenticated `session` mode.
`AI_PROVIDER`, `AI_PROVIDER_PRIORITY`	Assistant provider preference.
Provider key and model fields	Server-side credentials and an API model ID your account can access.
`AI_MAX_OUTPUT_TOKENS`, `AI_CONTEXT_CHARS`	Server response/context ceilings.
`TRUSTED_PROXY_HOSTS`	Explicit trusted proxy IPs/CIDRs; empty ignores forwarding headers.
`RATE_LIMIT_REQUESTS`, `RATE_LIMIT_TRAINING`, `RATE_LIMIT_ASSISTANT`	Abuse-control request limits.
`STORAGE_DIR`	Persistent source/model file location.
`MAIL_API_KEY`, `MAIL_FROM`, `PUBLIC_BASE_URL`	Optional email-verification configuration.

Native URL format:

```
postgresql+psycopg://USER:URL_ENCODED_PASSWORD@HOST:PORT/DATABASE
```

Generated passwords are URL-safe. URL-encode manually supplied passwords. Remote production PostgreSQL requires `sslmode=require`, `verify-ca` or `verify-full`; prefer `verify-full` with the provider's trusted CA and matching hostname. Include any required `sslrootcert` path using the provider's documented configuration. `require` encrypts transport but does not provide the

same hostname verification as `verify-full`. Preserve TLS verification during troubleshooting. Use the provider's host/port/database/application credentials, not assumed local defaults. Configure `COMPOSE_DATABASE_URL` separately when Docker should use that remote database.

AI configuration examples use field names, not credentials:

```
OPENAI_API_KEY=
OPENAI_MODEL=
GEMINI_API_KEY=
GEMINI_MODEL=
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=
```

Create an account with the provider you intend to use, enable API access/billing as required, generate a private API key and select a model ID listed as available to that account. OpenAI, Google Gemini and Anthropic use their corresponding fields above. For a compatible provider set `COMPATIBLE_API_KEY`, its exact HTTPS API root in `COMPATIBLE_BASE_URL` (often ending `/v1`, without `/chat/completions`) and `COMPATIBLE_MODEL`.

Fill at least one provider's actual key and usable API model ID locally, then restart the server. A coding application's model labels are not guaranteed API identifiers. Keys stay on the server. Check the provider's official account/API documentation for supported IDs and billing; no account access or commercial answer-quality result is bundled.

5. Projects, roles and invitations

A project keeps its datasets, evidence, models, scenarios, conversations and audit records separate. Use Projects in the header to return to the launcher. Choose a project according to its assigned role.

Operation	Owner	Trial researcher	Clinical reviewer	Viewer
Read data, graphs, predictions and exports	Yes	Yes	Yes	Yes
Import research data and protocol files	Yes	Yes	No	No
Run and save scenarios	Yes	Yes	No	No
Train, approve and activate models	Yes	No	No	No
Review protocol evidence	Yes	No	Yes	No
Use enabled assistant	Yes	Yes	Yes	No
Manage members/project settings	Yes	No	No	No
Inspect audit history	Yes	No	Yes	No

The server enforces these capabilities. The interface reflects them. The last owner cannot be removed or demoted. Password change invalidates other sessions; Account settings can sign out all sessions.

To invite someone: Settings, Members, Invite member; enter email/name/role and save. Share the resulting invitation link privately. The member opens it and creates a password. Existing accounts can be added directly. The interface states when no invitation email was sent. Invitation links expire.

6. Import and review data

Open More tools, Data Manager and load the starter cohort. Its 800 public ClinicalTrials.gov records and original source hashes are preserved. The five software-test fixtures are a separate dataset and cannot support scientific prediction or training.

Canonical imports require `nct_id` and `title`. Useful model fields are `phase`, `study_type`, `enrollment`, `number_of_arms`, `primary_outcomes`, `secondary_outcomes`, `criteria_count`, `brief_summary` and `eligibility_criteria`. Existing CSV/JSON, Excel, Parquet and documented registry/import workflows remain available.

Review quality, missingness and source hashes before analysis. Invalid numeric/date fields and duplicate study IDs cause a validation error without a partial import. Preserve raw AACT files unchanged. Arbitrary uploaded model binaries are not trusted saved models.

Duration fitting requires completed studies with precise start/completion dates and established ACTUAL completion outcomes. Estimated, imprecise, inverted, missing or nonpositive outcomes are excluded. Strict pretrial inference additionally needs consumed-feature availability timestamps.

Recruitment graphs require actual supported screening, eligibility, enrollment, follow-up/completion or site fields. Planned enrollment does not establish screening, retention or dropout counts. Absent fields remain unknown.

7. Overview graphs and study comparison

Overview aggregates the entire dataset. Phase/status filters apply to graphs and the study table; text search narrows the table. Source & method explains that distinction.

Choose Customize graphs, select your plots and Apply. Selection and ordering persist on this device for each project/dataset. Use arrow controls or drag handles to reorder. Fullscreen enlarges a graph; CSV/JSON export includes source values and methodology.

Graph	Meaning
Phase distribution	Counts across reported phase categories.
Status distribution	Counts across registry statuses.
Enrollment distribution	Reported enrollment in explicit bins, with missingness.
Protocol complexity	Fixed-reference PCI distribution and incomplete-input disclosure.
Enrollment vs complexity	Descriptive enrollment/PCI association.
Observed duration	Quartiles by phase from eligible ACTUAL exact-day outcomes.
Enrollment by phase	Reported enrollment totals and denominators.
PCI components	Weighted component summaries and missingness.
Observed trial dates	Registry dates with their original precision.
Missing data	Missing fractions/counts by field.
Start-year activity	Study counts by reported start year.
Model comparison	Saved held-out errors and complete metrics.

Select two or three study rows and Compare studies for a field comparison. Unavailable source measurements remain unavailable. PCI can be calculated with its documented zero-burden missing-component policy; partial coverage is disclosed and is not an outcome probability.

8. Saved-model prediction

Open Prediction Studio, choose Duration or Phase consistency, choose an approved saved model, select an evidence mode and Predict now. Saved preprocessing/models run on up to 100 validated records. Prediction does not trigger training.

Select a returned study to inspect values, intervals, explanation, source/model hashes and provenance. The phase pathway and prediction journey appear after actual evidence returns. Guided scenes explain these returned source/model/results; they do not invent phase transitions. Previous prediction runs can be reopened. Export CSV/evidence JSON or print to PDF.

Retrospective mode supports historical screening with source limitations. Strict pretrial mode checks supplied feature timestamps against the prediction cutoff; missing or late timestamps block the record.

Prolonged-duration probability concerns the training-defined duration threshold, not clinical success or delay relative to an operational plan. Split-conformal intervals may be unbounded or unavailable. The phase classifier estimates a phase label, not advancement probability.

9. Seven visual explanations

1. AI Pipeline connects the selected protocol, features, saved model and returned result. Job states reflect queued/running/completed/failed records; progress percentages are not invented.
2. Trial Timeline preserves reported date precision. Overall model-estimated completion requires an available duration and exact start date. Individual stage durations are not estimated.
3. Explainability exposes actual trained branches. Local additive waterfalls reconcile the intercept, signed contributions, exact remaining feature contribution and raw output. Linear contributions, global importance and fusion weights keep their method labels. Approved CatBoost branches provide native TreeSHAP. Constant baselines have zero feature effects.
4. Recruitment uses actual supported source counts and clearly unknown stages. Observed enrollment is separate from planned targets.
5. What-If Lab compares genuine saved-model predictions, keeping assumption-based planning separate.
6. Data Fusion shows actual structured/text encoders and available combination weights. Select a fitted branch; a constant baseline applies no feature encoders.
7. Research home's pointer-responsive helix is decorative and produces no biological prediction.

Each guided panel supports Play/Pause, Previous/Next, Replay and 0.5/1/2 speed, plus 2D/3D. Auto mode normally chooses 3D on desktop and 2D on mobile; explicit 2D stays available. Missing WebGL2, context loss or renderer errors fall back to accessible 2D. Exact values, units and source captions remain native HTML. Only one shared scene canvas renders at a time.

Application and operating-system reduced motion disable automatic playback while manual navigation remains available. Inactive, offscreen and background-tab scenes stop. Scene quality Low reduces rendering cost. Narration Off is the default; Narration On only enables the **Read aloud** button when browser speech is available. Speech starts only after that explicit click and stops on step/preference changes or when the panel becomes inactive. Browser voice availability varies by device.

Phase buttons explain educational categories; highlighted reported/model phase values come from the selected record and actual classifier. One study does not traverse all four phases.

10. Scenarios and planning

Open What-If Lab from More tools and choose a study and available model branch. Blank overrides preserve source fields. Changed fields apply only to the hypothetical copy; the original dataset remains unchanged.

Supply all planning assumptions: sites, participants per site per month, startup months and follow-up months. Name and run the scenario. Compare base/model results, changed/model results, PCI differences and explicit planning totals. A constant baseline can legitimately produce no prediction change.

```
Recruitment months = enrollment / (sites × monthly recruitment rate per site)
Planning total = startup + recruitment + follow-up months
```

This is arithmetic using your assumptions, not a trained recruitment-risk forecast. Protocol perturbations are not causal evidence. The Scenarios workspace retains saved comparisons and provenance. Network errors never trigger a substitute prediction.

11. Train, compare, approve and activate

Owners can open More tools → Experiments, review readiness, select Include CatBoost, choose Standard search or Expanded CatBoost search and Run experiment. Actual run states and completed manifests/held-out predictions are retained. Researchers/reviewers/viewers cannot train or activate.

Standard search (the default/basic mode) preserves the v5.2.0.2 CatBoost parameters and two ordered depths, 4 and 6. Expanded search evaluates eight ordered combinations per task: depths 4/6, L2 values 3/10 and 120/300 iterations. It needs more fitting time and does not guarantee better held-out results. CatBoost must be enabled separately. One-class training can use the existing prior fallback and records only searches actually performed.

Each candidate fits preprocessing and estimator on training records only. Duration selection uses tuning RMSE; risk selection uses tuning Brier. Calibration and test records do not fit/search candidates. Exact ties keep the first declared candidate. The manifest records configurations, actual/effective parameters, trial scores, selected indices, split counts, tie rule and CatBoost version. No validation refit or automatic activation occurs.

The nine existing branches are median/prevalence baseline, structured/text/multimodal linear models, Extra Trees, Random Forest, Gradient Boosting, Histogram Boosting and weighted late fusion. Opt-in CatBoost adds a tenth candidate and native additive TreeSHAP.

Preprocessing fits training records only. Tuning, calibration and test splits retain distinct roles; date groups and outcome-availability purging protect direct split separation. Compare duration MAE/RMSE and classification Brier/AUROC/average precision. Undefined metrics remain unavailable.

The preserved v5.2.0.2 frozen-cohort comparison used 800 inputs, 312 eligible precise ACTUAL outcomes, 123 temporal exclusions and retained splits of 105 training, 9 tuning, 11 calibration and 64 test records. The median baseline remains the duration validation winner. CatBoost is a candidate capability, not an activated accuracy claim. Detailed evaluation evidence is included in `docs/v5202-evidence/`.

The v5.2.0.3 authentic 800-record expanded comparison kept the prior basic run's splits and recommended `text` for the risk branch in both runs. CatBoost duration validation RMSE was `583.6089521513263` and test RMSE was `454.49926978644356`, unchanged from basic. Tuning Brier changed from `0.08050878086835794` (basic) to `0.07927376884169636` (expanded), while held-out test Brier changed from `0.11011105593329391` to `0.11806014427359898` (worse). These exact reported results do not support an overall accuracy improvement. The candidate was not activated and approved artifact selection remains unchanged. Read the current release evidence for complete manifests and methodological limits.

After completion, open Models. Inspect source, metrics, hashes, eligibility and limitations. Owner approval is required before activation. Training alone does not replace the active model. Branch selectors list branches present in the approved artifact; CatBoost appears after a compatible candidate is approved/activated. Do not repeatedly tune on the test set.

12. Protocols, sources and formulas

Protocol Evidence imports supported PDF/text, extracts text and links fields to page/section evidence. Inspect excerpts before using extracted values. Owners/clinical reviewers can review or correct evidence; researchers can import documents. Scanned PDFs without extractable text need a supported separate text/OCR workflow.

Version comparison identifies added/removed text. Amendment-risk prediction is unavailable without suitable labels and validation. The advanced reviewed rubric is a separate user-defined calculation.

Research sources integrates ClinicalTrials.gov, PubMed and openFDA. Save useful results to the project evidence library. Live services need internet and can report availability errors. Complexity Lab and Formula Studio explain fixed PCI weights, ceilings, rounding and missing-data policy.

Phase Academy, Phase Model, Research Hub and Validation Lab retain their existing research/educational workflows and saved evaluation evidence.

Fixed-reference PCI, with explicit assumptions

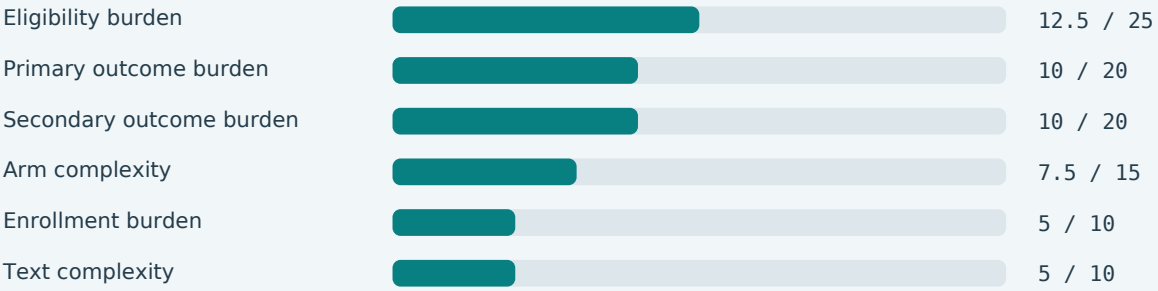
```
PCI = sum(weight_i × clip(value_i / ceiling_i, 0, 1))
```

Component	Input field	Ceiling	Weight
Eligibility burden	`criteria_count`	40	25
Primary outcome burden	`primary_outcomes`	10	20
Secondary outcome burden	`secondary_outcomes`	20	20
Arm complexity	`number_of_arms`	8	15
Enrollment burden	`enrollment`	1000	10
Text complexity	`text_complexity`	20	10

Sum unrounded components, then round the total to two decimals using exact decimal half-up. Categories use the rounded score: Low below 40, Medium 40–70 inclusive, High above 70. Missing, invalid and nonfinite inputs contribute zero and are disclosed, so incomplete records may be understated. Finite negative values clip to zero. These are descriptive research assumptions, not clinically validated thresholds or probabilities. Exploratory custom weights and training-relative research PCI retain separate method labels.

A formula-only check sets each input to half its ceiling, producing 50 points. That arithmetic example is not a trial record or clinical result.

PCI known answer: every value at half its ceiling gives 50 points



Bars use one common 0-25 contribution scale. These are formula inputs, not trial outcomes.

13. Assistant configuration and feedback

Configure one usable provider key/model ID in the private server environment and restart. OpenAI, Gemini, Claude and compatible HTTPS services are supported. Use Settings, Assistant for provider/profile, enable switch and budgets. A daily quota of 0 means no daily application quota; request and provider limits still apply.

The composer accepts questions up to **32,768 characters**. General questions can receive a clearly separated general-knowledge answer without unrelated project citations. Project-specific statements require the supplied project evidence and exact source references. Context cards identify supplied evidence; they are not proof of every sentence in an answer.

New projects store daily application quota **0**, meaning no daily application quota. Existing explicitly configured quotas remain unchanged; older omitted settings can retain the legacy fallback. Provider billing/rate/context/output ceilings, project budgets and request abuse controls still apply. This is not unlimited provider usage or an accuracy guarantee.

Open Assistant, choose a study or dataset context and send a question. Replies stream as readable Markdown, including lists/tables/code. Sources identify actually supplied references. Copy/export preserves answers. Errors do not create invented replies.

Context includes actual cohort quality, selected-study fields, saved predictions, active-model evaluation, protocol excerpts and relevant saved evidence. Verify answer claims against their sources. Commercial provider access needs your own usable credentials; automated provider and UI streaming checks use local test responders.

Useful/Needs review records feedback against a completed message owned by your account. An owner uses Settings, Feedback review to approve an evidence-backed correction. Approved guidance can inform later project prompts. Pending feedback is not an instruction and source measurements take precedence.

This is reviewed context improvement. Model weights change through evaluated training and owner approval; feedback does not prove increased accuracy or automatic retraining.

14. Settings and security

Account settings contain email verification and password controls. Visual settings stay focused on appearance and guided scenes.

Settings groups Appearance, Visual explanations, Project, Members, Assistant, Audit, Feedback and Account according to your role. Appearance offers Light/Dark/System, reduced motion, chart labels/grid and Comfortable/Compact spacing. Visual explanations offers Auto/2D/3D, Low/Standard quality and Off/On narration with a conceptual preview. Changes save on this browser. Export/import/reset handles validated appearance preferences only, not credentials, accounts or project data. Old valid appearance choices migrate; unknown settings are discarded.

Graph layouts remain device-local per project/dataset. Within an authenticated session, reload restores the authorized project, route, dataset, selected study and filters. Fresh sign-in shows Projects. Explicit Projects navigation returns to the launcher. Logout, expiry, identity change, deleted project or membership loss clear workspace intent; server permissions remain authoritative.

Project assistant settings include enablement, provider/profile, response/context ceilings, daily quota and retention. Members and audit use real server records. Account controls change passwords and sign out sessions. Reducing retention can remove old conversations. Refresh after a membership change.

PostgreSQL schemas isolate project metadata. Feedback IDs never become filenames. Request limiting covers research forwarding and uses configured trusted-proxy policy. Startup disables Uvicorn's automatic forwarding-header trust.

15. Automated and manual tests

Use a separate disposable PostgreSQL database, never your live research database. Create a test database and configure its private `TEST_DATABASE_URL` in `.env`; the test role must be permitted to create/drop its temporary schemas. Native users can use a separately named local test database without Docker. Compose's optional test profile runs separately on port 5433. Tests use private run-scoped schemas and remove them after the run. Read chapter 22 for the release's actual verification status.

```
docker compose --profile test up -d postgres-test
```

Windows:

```
py -3.12 scripts/setup.py --dev
.venv\Scripts\python.exe scripts/run_tests.py
.venv\Scripts\python.exe scripts/run_smoke_test.py
.venv\Scripts\python.exe scripts/verify_v5_models.py
```

Linux/macOS:

```
python3.12 scripts/setup.py --dev
.venv/bin/python scripts/run_tests.py
.venv/bin/python scripts/run_smoke_test.py
.venv/bin/python scripts/verify_v5_models.py
```

Frontend:

```
cd frontend
npm ci
npm run build
node --test src/components/animations/evidence.test.mjs
  src/components/animations/components.test.mjs
```

These commands are reproducibility instructions, not a claim that your installation or the current release has passed. The suite checks PostgreSQL, auth, roles, isolation, imports, inference, scenarios, actual explanations, cohort graphs, fitting, feedback, grounding and provider/stream error handling. An optional external SHAP-library test may skip if that dependency is absent; native CatBoost TreeSHAP has separate executed tests.

Manual acceptance: use the 37-check workflow in chapter 19. It covers create/login/open a project; load data; select/filter/reorder/export charts; compare studies; predict and inspect phases; run a scenario; inspect explanations/fusion; import/review a protocol; verify each role; change appearance/account settings; test assistant feedback. Commercial provider/email/remote deployment checks need operator configuration.

16. CLI research and release builds

A standalone fitting example:

```
.venv/bin/python scripts/train_models.py --input data/starter/trials.csv --output
  results/my-comparison --config configs/research-catboost.json --metadata
  data/starter/metadata.json
```

On Windows use `.venv\Scripts\python.exe`. Output must be a new directory. The CLI exports hashes, manifests, exclusions, predictions and metrics; it does not create a project approval or activate a model. `configs/research.json` retains the original candidate set.

For expanded CatBoost use `configs/research-catboost-expanded.json` instead of the basic `configs/research-catboost.json`. Keep every run in a new output directory. Compare eligibility, retained split IDs, configuration and actual held-out metrics before considering activation. The new search does not alter the historical approved artifacts.

Optional BioBERT/survival methods require their separate dependencies and appropriate cohorts; they are not newly trained release models. Ongoing/censored outcomes require a separate survival analysis.

Rebuild a release:

```
npm --prefix frontend run build
.venv/bin/python scripts/package_release.py --version 5.2.0.3 --output
  ../Clinical-Trial-Intelligence-Workbench-v5.2.0.3.zip
```

The archive includes current built assets and portable paths/checksums. Secrets, runtime databases, dependency folders and nested release ZIPs are excluded.

17. Backup and deployment

Back up PostgreSQL and research-file volumes together. Database records do not contain all PDFs/CSVs/model binaries. Use your provider's backups or PostgreSQL `pg_dump`; keep your private environment secure.

Deployment requires HTTPS, authenticated session mode, production values, TLS PostgreSQL, persistent storage and explicitly trusted reverse proxies. The release validates a development/container workflow, not a certified production or clinical deployment.

For a previous SQLite development installation, export/import source datasets and retain hashes. This release does not automatically migrate SQLite or ship old runtime databases.

18. Troubleshooting and evidence limits

Symptom	Check
Python launcher unavailable	Install Python 3.12 and its launcher; verify `py -3.12 --version`.
Native database fails	Start the PostgreSQL Windows service; verify installation port, local role/database and private credentials; run connector `--check`.
Existing-role password differs	Enter its known application password or choose a new role; the connector will not reset it.
3D unavailable or slow	Select 2D or Low quality; captions and original views remain available.
Narration silent	Enable narration, click Read aloud and check browser voice support; it never starts automatically.
Reload opens Projects	Verify active authentication and membership; fresh sign-in/explicit Projects/expiry intentionally use the launcher.
Docker fails	Docker Desktop running, Linux engine, Compose available.
Port conflict	Change host port and relevant URL together.
Database failure	Database running; correct host/port/credentials/database/TLS.
Setup token rejected	Use local generated token; setup closes after an owner exists.
Invitation fails	Correct project/email and an unexpired invitation.
Prediction unavailable	Real compatible data and an approved intact artifact.
Pretrial blocked	Cutoff and consumed-feature availability timestamps.
Recruitment unknown	Actual supported source counts, not assumed retention rates.
No scenario model change	Constant model or unsupported input; select a fitted branch.
Assistant unavailable	Usable key/model ID, enabled assistant, permitted role.
Reply incomplete	Larger supported budget/profile or narrower question.
Request limited	Wait for the rate window; quota 0 does not disable abuse controls.
Old interface appears	Rebuild `frontend/dist` and restart/reload.

The cohort is retrospective and becomes small after outcome/split filtering. Feature snapshots may include post-start updates; the nine-record tuning split does not support strong generalization claims. Clinical-success, treatment-effect, recruitment-risk and amendment-risk probabilities are not validated bundled tasks. Inspect source records, explicit assumptions and evaluation evidence before interpreting research outputs.

Local developer mode (`AUTH_MODE=local`) bypasses sign-in and therefore does not support attributed assistant feedback. Use the default `AUTH_MODE=session` for feedback, owner review and team access.

19. All sections and the 37-check acceptance workflow

The eight core navigation items and More tools retain **26 routes**. Settings may hide operations that your role cannot perform. Check each section with actual imported data; record Pass, Fail, Unavailable or Not run with a date and evidence. These are acceptance steps, not prefilled results.

Check	Section or workflow	Expected behavior
1	Overview	All 12 graph choices, filters, ordering, fullscreen and CSV/JSON methodology work on the full cohort.
2	Studies	Search, phase/status, sort, paging, actual detail and two/three-study comparison preserve source fields.
3	Prediction Studio	Bundled approved duration/phase prediction returns actual results, provenance and saved history.
4	Protocol Evidence	Supported text/PDF import retains excerpts; permitted review and version comparison remain available.
5	Scenarios	Saved hypothetical inputs, real model differences and separate planning assumptions remain labeled.
6	Models	Artifact hashes/metrics, approval and explicit owner activation are visible; no automatic activation.
7	Assistant	Authorized configured users receive a completed stream; sources, copy/export and feedback remain available.
8	Settings	Appearance, scenes and role-permitted project/account functions work and persist appropriately.
9	Research home	Import → Review → Predict → Interpret → Compare links open existing routes; helix is decorative.
10	Research sources	ClinicalTrials.gov, PubMed/openFDA results and upstream errors remain explicit.
11	Saved library	Saved source evidence is project-scoped and readable.
12	Research Hub	Existing linked research workflows and selected-study handoffs remain available.
13	Complexity Lab	Returned PCI components and missingness match the documented fixed formula.

Check	Section or workflow	Expected behavior
14	Formula Studio	Formula, ceilings, weights, rounding and assumptions are explained without outcome-risk claims.
15	Experiments	Owner can queue basic/expanded fitting and inspect actual run status/manifests.
16	Validation Lab	Existing saved evaluation and readiness limits remain available.
17	Phase Academy	Optional 0 and I–IV, all four lesson steps, quiz, recorded examples and primary sources remain.
18	Phase Model	Owner training and saved consistency evaluation remain labeled; no advancement probability.
19	Data Manager	Load the authentic 800 cohort; supported imports validate atomically and preserve source hashes.
20	AI Pipeline	Actual selected model and job status appear; educational architecture remains conceptual.
21	Trial Timeline	Exact/month/year precision and ACTUAL/ESTIMATED type remain visible; no invented phase timings.
22	Explainability	Local additive evidence reconciles intercept, signed effects, remainder and raw total; other methods retain labels.
23	Recruitment	Reported participants, planned targets and sites retain distinct units; missing stages remain unknown.
24	What-If Lab	Baseline/scenario predictions are genuine; planning months stay separate from model days.
25	Data Fusion	Only actual available branches/encoders/weights are shown.
26	Access Control	Owner-only member/role management; last-owner and server permission protections remain.
27	Fresh Windows setup	Python setup, native connector and start work on extracted ZIP; hidden admin password is never saved.
28	Database repeatability	Check/reconnect preserves existing role password, sentinel data and unrelated environment settings.
29	First owner and Projects	Local setup token creates the first owner; create/open project and return to Projects.
30	Refresh and identity	Reload restores authorized context; logout, expiry and lost membership clear it safely.

Check	Section or workflow	Expected behavior
31	Role isolation	Owner/researcher/reviewer/viewer actions match the server capability table; projects remain isolated.
32	Settings portability	Light/Dark/System, density, chart options and valid preference import/export/reset work.
33	Scene controls	2D/3D, manual steps, replay/speed and exact HTML values work; only one scene canvas is active.
34	Motion and fallback	App/system reduced motion, offscreen/background stop and unavailable WebGL keep usable 2D.
35	Narration and mobile	On enables explicit Read aloud; Off removes it; mobile Auto uses 2D with readable controls.
36	Assistant limits/review	32,768-character input boundary, general/source distinction, quotas, errors and owner-reviewed feedback work.
37	Exports, restart and release	Open exported reports/CSV, restart and recover projects; verify ZIP/extracted checksums and exclusion of secrets.

An offline/static preview is a separate interface demonstration. It does not run live imports, model jobs, PostgreSQL access or commercial providers. Use the complete local application for end-to-end checks. Do not substitute the five software fixtures for the frozen public cohort.

20. Module and file map

Location	Responsibility
SETUP_WINDOWS.bat, scripts/setup.py	Python 3.12 virtual environment and private environment preparation.
CONNECT_DATABASE_WINDOWS.bat, scripts/configure_postgres.py	Native PostgreSQL prompts, create-missing-only provisioning and verified connection.
START_WINDOWS.bat, scripts/start.py, scripts/doctor.py	Native startup and credential-safe diagnostics.
docker-compose.yml, Dockerfile, Docker launchers	Optional container deployment and separate test service.
backend/app/	FastAPI endpoints, PostgreSQL platform/auth, server configuration and research/assistant services.
src/clinical_trial_intelligence/	Ingestion, research pipelines, training-only preprocessing, validation and inference.
frontend/src/App.tsx, v5/, v52/, v5202/, v5203/	Workspace composition, platform, prediction, research UI, navigation and current visual/session layer.
frontend/src/components/three/	Shared lazy scene shell/canvas, exact evidence captions, motion and fallback controls.
frontend/src/components/animations/, education/	Evidence adapters, existing 2D explanations and Academy/home integration.
frontend/dist/	Prebuilt browser assets used by the ZIP; rebuild only after frontend edits.
configs/	PCI/shared formulas and reproducible basic/expanded experiment configurations.
data/starter/, data/raw/, data/sample/	Frozen public starter records, preserved raw inputs and separate software fixtures.
data/workbench/ or STORAGE_DIR	Private persistent imports/protocols/models; back up with PostgreSQL.
models/, examples/	Saved approved artifacts and historical reproducible research examples.
tests/, scripts/run_tests.py	Backend/native connector regression and PostgreSQL integration checks.
docs/v5202-evidence/, docs/v5203-evidence/	Historical and current release evidence; their release contexts remain distinct.
docs/v5.2.0.3-full-guide.md, scripts/build_v5203_guide.py	Editable operating guide and deterministic ReportLab PDF builder.
scripts/package_release.py, scripts/verify_release.py	Portable archive creation and packaged checksum verification.

The current guide does not overwrite historical v5.2.0.2 documents. Consult those only for their release's evidence and older behavior. Application records live in PostgreSQL; source files and artifacts live on disk, so one backup alone is incomplete.

21. What changed from v5.2.0.2

Area	v5.2.0.3 change
Native Windows installation	Dedicated connector permits PostgreSQL without Docker and hidden-password provisioning of missing app objects.
Visual design	Manrope typography, stone/cool-white surfaces, emerald/slate/indigo/coral accents and clearer spacing replace the heavier textured navigation.
Guided explanations	Shared lazy 3D for phases, pipeline, attribution, recruitment, PCI, fusion, dates, scenarios and research journey; 2D values stay available.
Academy/home	Preserved phase lessons/quizzes/sources plus guided controls and research journey; helix remains explicitly decorative.
Session restoration	Reload recovers only an authorized active workspace; identity/membership changes invalidate stale browser intent.
Settings	Scene mode/quality, narration and density; validated browser preference export/import/reset and grouped controls.
Assistant	Input limit increases to 32,768 characters; general knowledge separated from project claims; new-project quota 0; existing limits preserved.
Optional model fitting	Expanded CatBoost search records actual ordered trials and tuning-only selection; basic/default behavior remains compatible.
Release materials	Current operating guide/PDF, verification report and evidence are separate from preserved historical documents.

All 26 routes, raw records, fixed PCI, approved artifact selection, source hashes, role enforcement and historical research evidence remain part of the workbench. Additional rendering or candidates do not establish better clinical outcomes or validated predictive accuracy.

22. Verification status and downloadable materials

Release validation passed 466 PostgreSQL-backed Python checks, 58 frontend checks, TypeScript/production build, 9 SSE parser checks and 25 authenticated real-data browser checks. Fresh/stale session, WebGL absence, context loss, reduced motion and delayed cross-dataset responses were also checked. Saved model outputs and point metrics replayed exactly within numerical tolerance. Native PostgreSQL helper lifecycle checks passed. Read `docs/v5.2.0.3-release-verification.md` for evidence and limitations; extracted ZIP/public-download integrity is checked against the frozen packaged files. Windows GUI operation and commercial-provider answer quality remain operator checks.

Windows GUI execution, real commercial provider accounts/answer quality, optional email service, managed remote PostgreSQL/TLS and production infrastructure require their corresponding operator environment. Linux-executed connector tests do not establish Windows GUI acceptance. A local test responder can demonstrate streaming integration without establishing commercial LLM accuracy.

Open `docs/Clinical-Trial-Intelligence-v5.2.0.3-Full-Guide.pdf` directly, or read this editable Markdown. The complete release ZIP includes source, built UI, data/artifacts, setup helpers and guide; its filename identifies v5.2.0.3. Use the delivered public download link rather than a localhost address. Verify the delivered archive SHA-256 against the published checksum, extract it and run the supplied archive/file verifier before installation.

Download integrity and PDF reproduction

Windows PowerShell, before extracting the delivered archive:

```
Get-FileHash -Algorithm SHA256 .\Clinical-Trial-Intelligence-Workbench-v5.2.0.3.zip
```

Compare the result with the separately published archive digest. After extraction, run from its project root:

```
py -3.12 scripts/verify_release.py
```

Linux/macOS use `sha256sum` (or `shasum -a 256`) for the ZIP and `python3.12 scripts/verify_release.py` inside the extraction. The packaged `SHA256SUMS.txt` validates file integrity; its bundled checks alone do not independently authenticate the download.

To rebuild only this guide, install the optional documentation requirements in your Python environment and run:

```
.venv\Scripts\python.exe -m pip install -r requirements-docs.txt
.venv\Scripts\python.exe scripts/build_v5203_guide.py
```

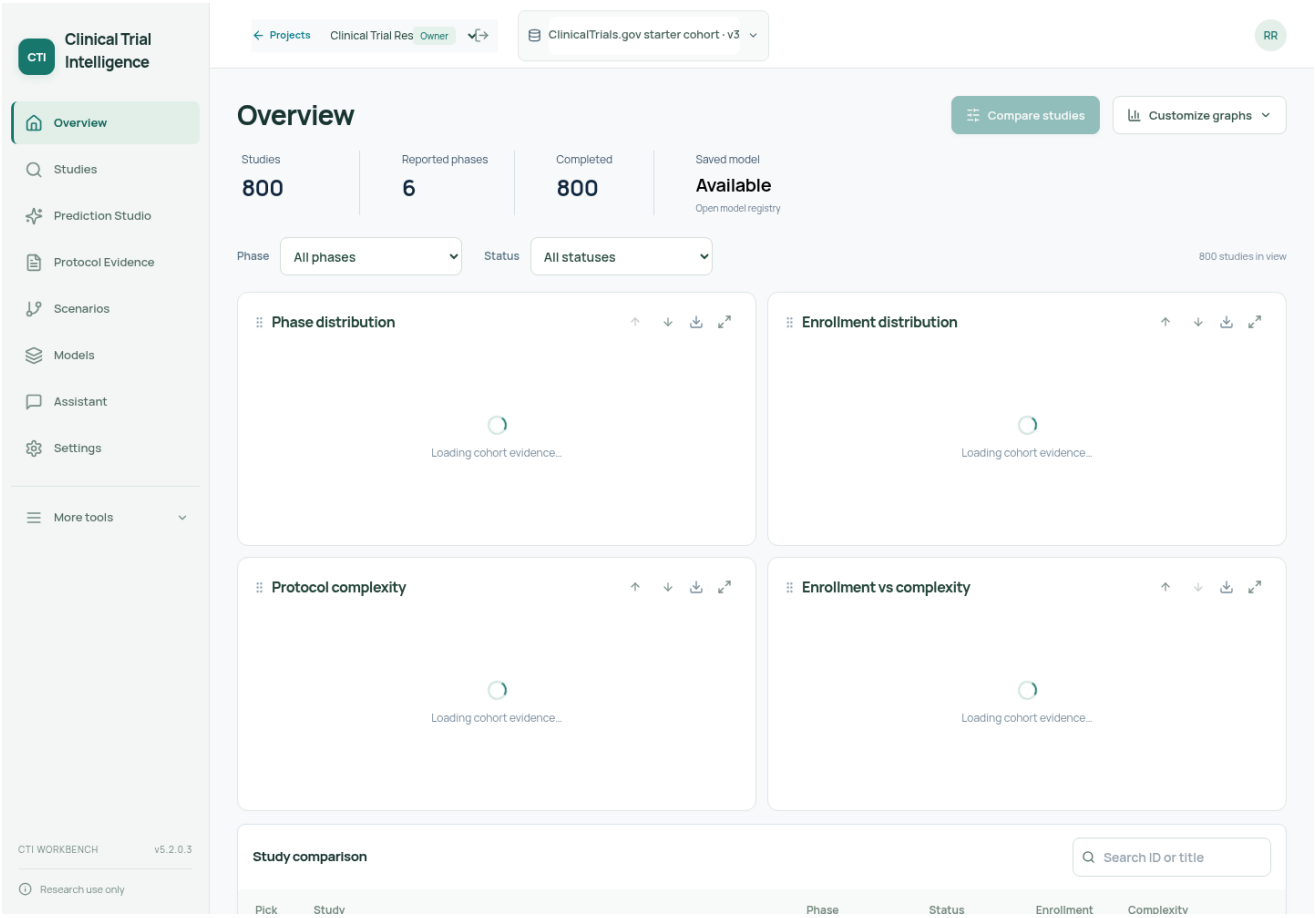
Linux/macOS substitute `.venv/bin/python`. The builder reads this Markdown and includes available actual screenshots under `docs/v5203-evidence`; it makes no provider/application calls. End users can simply open the included PDF.

Historical approved metric strings are preserved in their original result files and guides. Never label software-fixture scores as cohort results or describe an unactivated candidate as the approved model. Research decision support is not clinical or regulatory advice.

23. Actual application screenshots

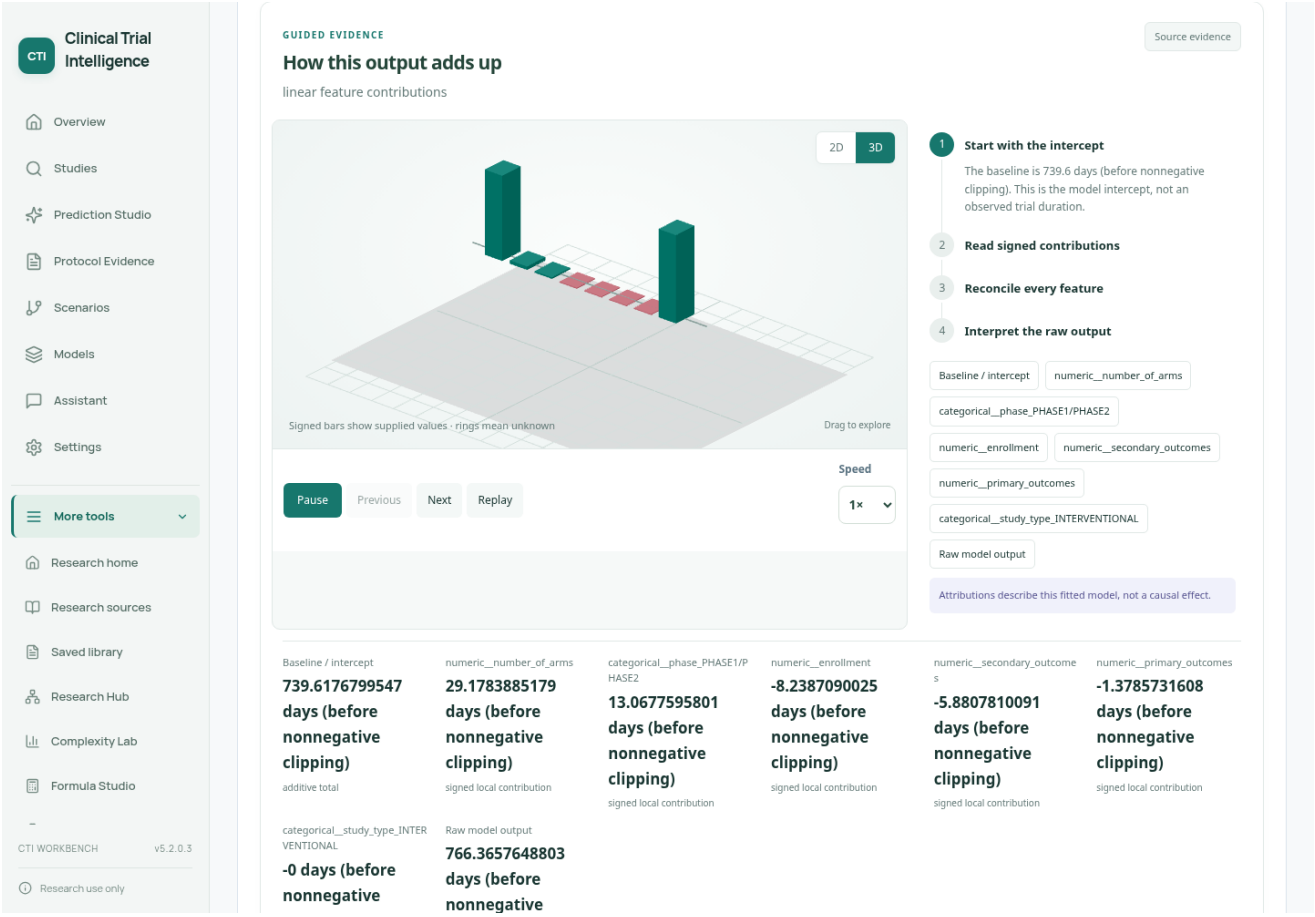
Screenshots captured from the application during release verification. Read the accompanying report for exercised workflows and limits.

Overview · full-cohort research workspace



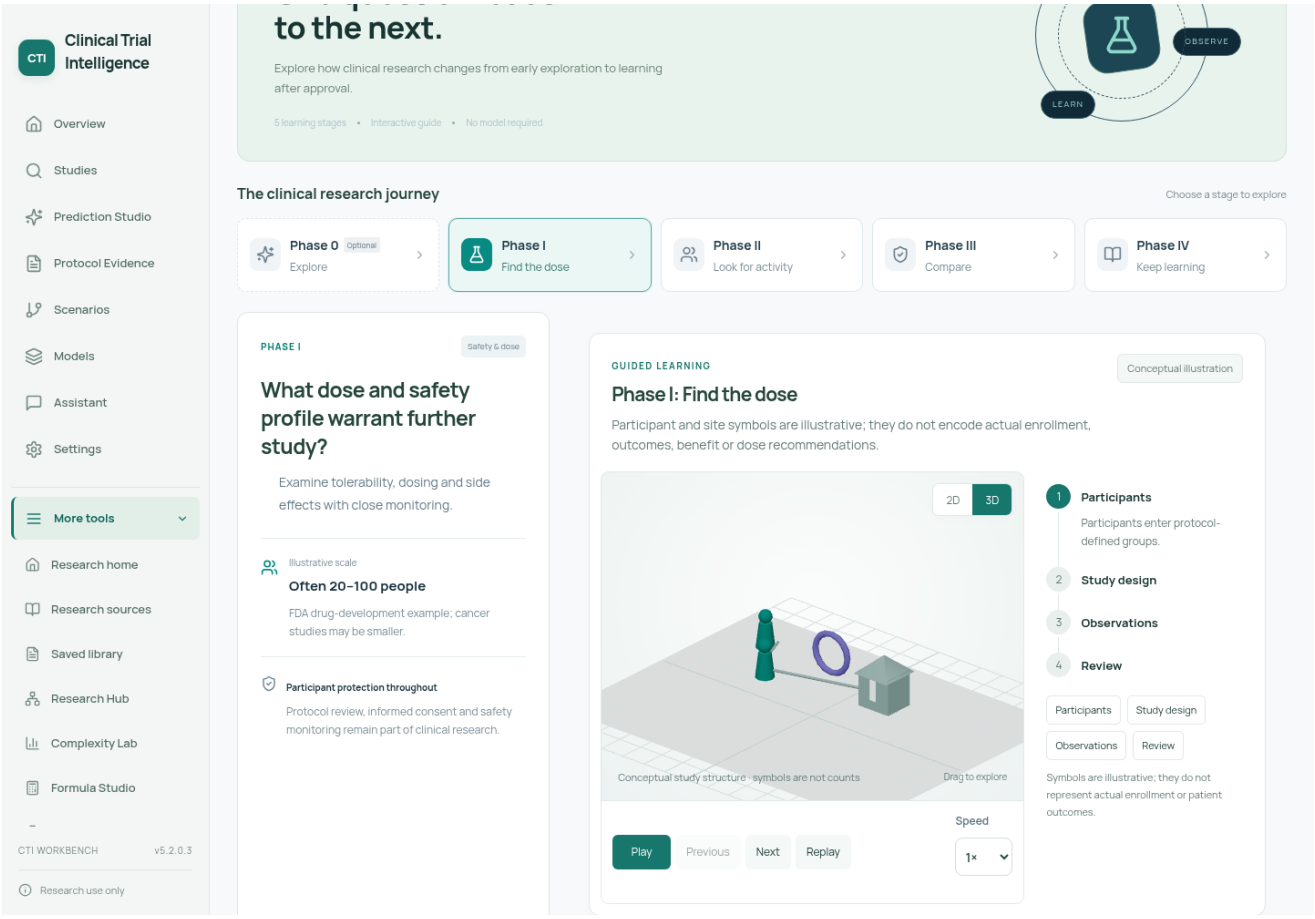
overview-desktop.png

Explainability · actual attribution and guided evidence



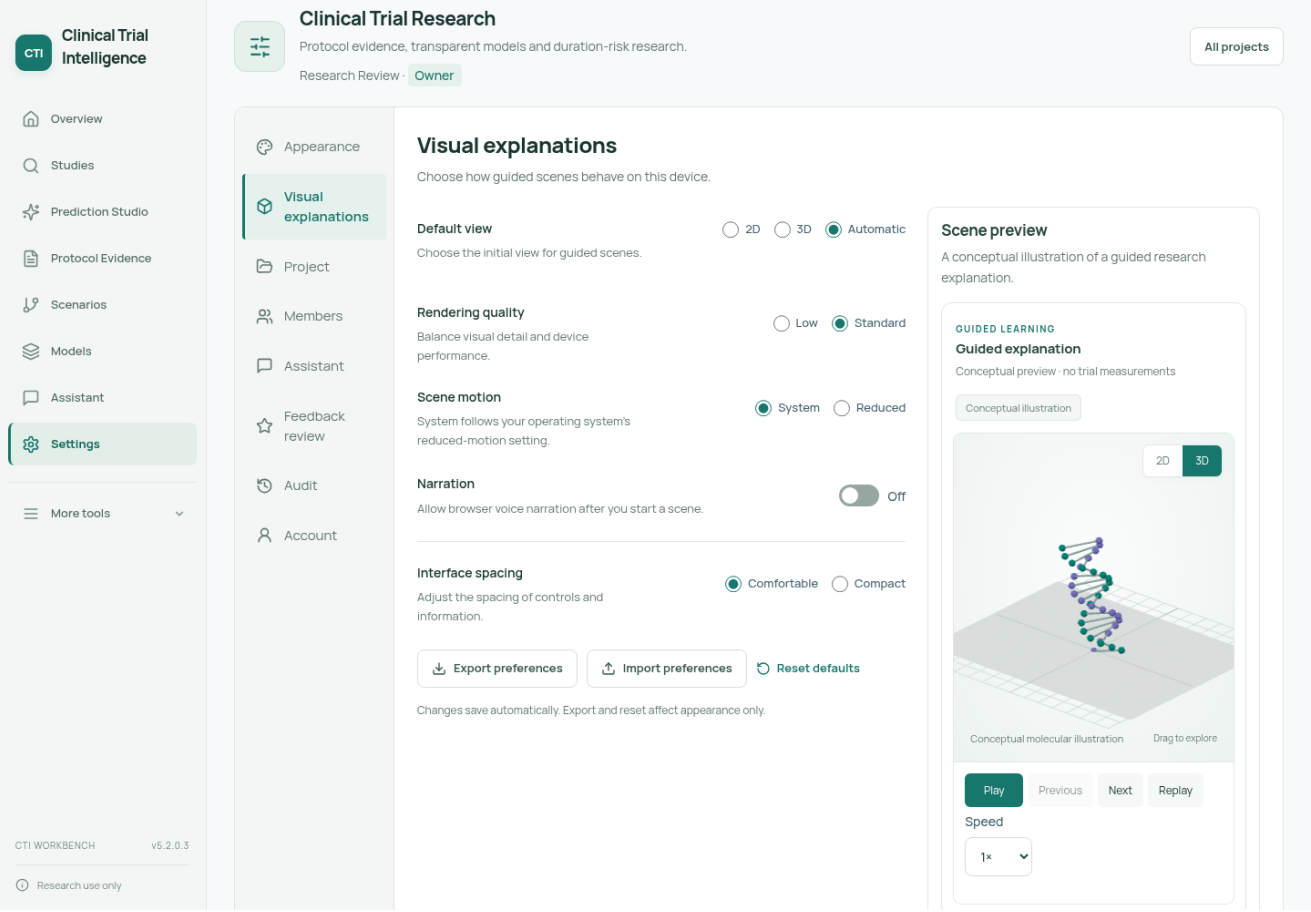
xai-desktop.png

Phase Academy · preserved educational lessons and scene controls



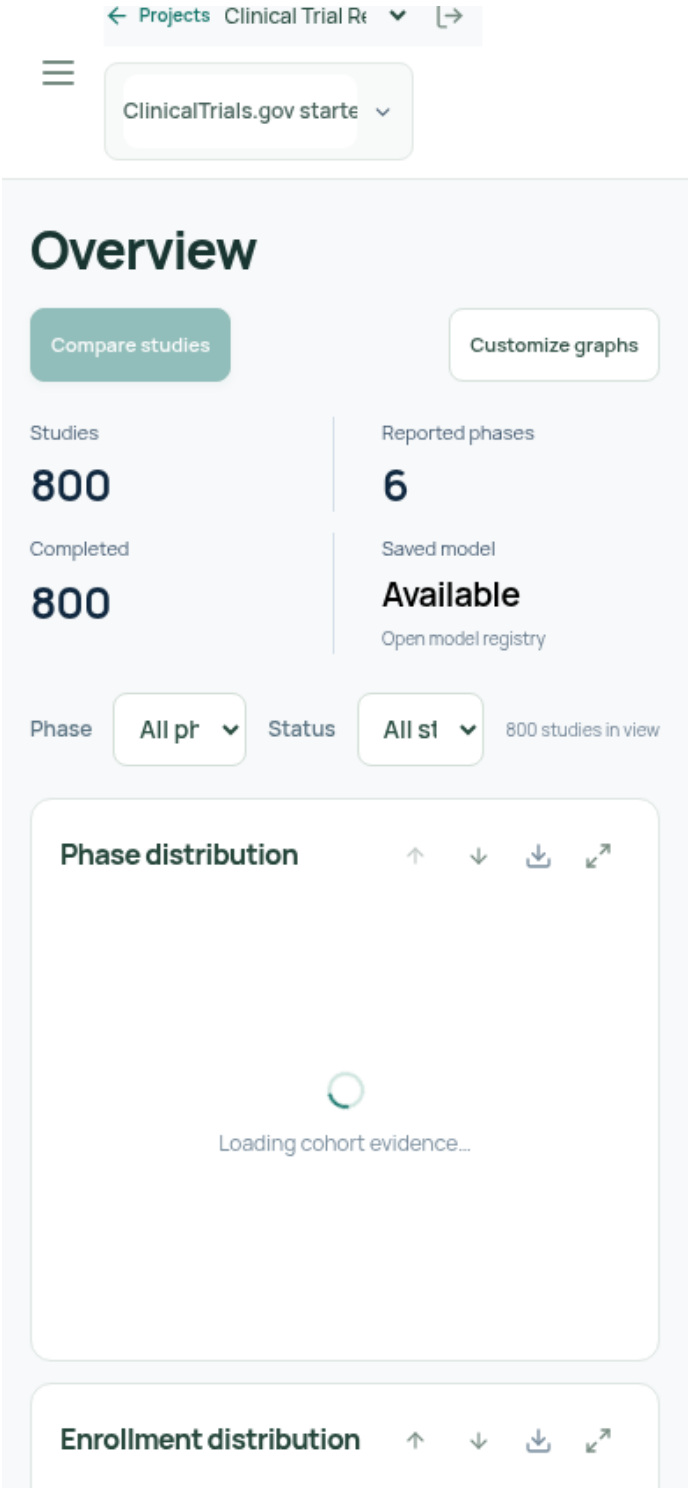
academy-desktop.png

Settings · browser appearance and visual explanations



settings-desktop.png

Mobile · responsive research workspace



overview-mobile.png